The Open Source Audit

A Capstone Project for OSS NGMC Course

Name: Shree Nigam
Registration Number: 24BAI10688
Slot: A24

Submission Date: 29/03/26

Chosen Software: Python with Linux

Part A — Origin and Philosophy

A1. Why Python Was Born

Back in the late '80s, writing code was honestly pretty tough. If you wanted power, you had to slog through languages like C—fast, but not easy to read or use. For quick fixes you could turn to Bash scripts, but good luck building anything big or complex with those. Guido van Rossum, working at CWI in the Netherlands, ran into these problems while wrestling with the Amoeba operating system. He wanted something in between: a language that cared more about the developer's time than shaving milliseconds off execution. It had to be readable—almost like English—but not wimpy, so he could use it for real work.

Over Christmas in 1989, Guido started sketching out a new interpreter. He took cues from a teaching language called ABC, but made it useful with features like exception handling and real data structures. He didn't want to hoard his creation or sell it off. Instead, Guido dropped the code publicly in 1991 because, in academia and hacker circles, sharing means faster progress, better feedback, and a thriving community. The name? Python—not for the reptile, but after Monty Python, the comedy group. He wanted coding to feel fun, not a grind.

A2. The License

Open source means more than just free downloads. The four big freedoms are: run it however you want; study and change it; share it; and share your tweaked versions. Python's license (the PSF License) checks all these boxes—it's permissive, recognized by the OSI, and doesn't insist you keep your changes public. So, a company could take Python, rework it for their own proprietary needs, and sell it, as long as they keep the original copyright info.

This is where it really differs from the GPL. GPL forces all derivative works to stay open, while the PSF and MIT licenses let developers lock down their code if they want. If I built a tool aiming for massive industry adoption, I'd pick MIT or PSF for maximum flexibility. If I was fighting corporate monopolies, I'd go GPL.

Python really cuts to the heart of "free beer" versus "free speech." Downloading Python for zero cost is "free beer." But being able to see the source, hack out the garbage collector, drop in a new one, and rebuild Python for a supercomputer—that's "free speech," real freedom.

A3. The Ethics of Open Source

Should all software be open source? That's never simple. On one side, open code means faster innovation—people learn from each other, vulnerabilities get fixed globally instead of by a locked-down team. On the flip side, things like medical imaging software or pro video tools require lots of investment. If companies can't recoup costs through licenses, those projects might not ever happen.

Then there's the corporate angle. Huge companies profit from free community labor—Linux servers, OpenSSL, Python libraries, you name it. Some call it exploitation, but often it's mutual. Those same corporations sponsor open source foundations, pay maintainers, and keep the ecosystem healthy.

When developers say they're “standing on the shoulders of giants,” they aren’t kidding. Python itself rests on C. Modern AI owes everything to Python. Open source lets everyone start from the cutting edge, not from scratch.

Part B — Linux Footprint

I wanted to see how Python fits into a typical Linux setup, so I poked around my own environment. Python's everywhere—almost every package manager has it ready to go. On Ubuntu systems, installing Python is as easy as:

```
sudo apt install python3
```

RedHat uses dnf or yum. So many basic tools (including apt itself!) rely on Python, it usually comes pre-installed.

You'll find the main Python binary at /usr/bin/python3. Tons of built-in modules—the "batteries" that make Python famous—sit in /usr/lib/python3.x/. Unlike Apache or other daemons, Python doesn't have a big config file in /etc/ because it's just an interpreter. But pip configs and related scripts can end up in /etc/ or your home directory.

Security-wise, Python scripts run as whoever called them. If I run python3 script.py, it has the same access as me. Run it with sudo, and it's got root privileges—which is why grabbing random scripts from the web and running them as root is a terrible idea.

Python updates alongside your Linux system. When a security bug appears, the PSF releases a patch, maintainers pack it up, and it lands on your machine the next time you update.

Part C — The FOSS Ecosystem

Python isn't floating in a vacuum. It's surrounded by a gigantic ecosystem—the interpreter itself is built with GCC, and depends on stuff like OpenSSL (for networking), libffi (for foreign function calls), and glibc.

On top of this foundation, Python has powered an array of revolutionary projects. It's been the go-to language for AI—helping launch frameworks like PyTorch, TensorFlow, and Scikit-Learn. Web development? Python's everywhere, running massive sites with Django, Flask, and FastAPI. It even inspired new implementations like PyPy (with JIT compilation) and MicroPython (for microcontrollers).

In the classic LAMP stack, Python can step right in for the "P" slot, often replacing PHP. Lots of modern sites now run Linux, Nginx, PostgreSQL, and Python (the so-called LEMP variant), serving millions daily.

The PSF (Python Software Foundation) acts as the steward for the language. Changes happen transparently through Python Enhancement Proposals (PEPs). For decades, Guido van Rossum personally guided the language as "Benevolent Dictator For Life." Nowadays, a community-elected steering council keeps things democratic and open.

Part D — Open Source vs Proprietary

Comparing Python to MATLAB (a proprietary alternative):

• Cost? Python's totally free. MATLAB demands pricey licenses for every user.

• Security? Python's source code is open for anyone to check. MATLAB? Only the vendor gets to see the internals.

• Support? Python has a massive community and tons of StackOverflow hits. MATLAB offers official paid support.

• Freedom to modify? Python lets you fork, tweak, rebuild—anything really. MATLAB keeps all that locked away.

• Control? Python's run by a nonprofit foundation. MATLAB's a corporate product, focused on selling to enterprise.

Looking over all this, it's clear Python is a smart pick for real-world projects. Sure, it has limits—if you're building ultra-fast trading systems or self-contained desktop apps, you'd reach for C++ or something else. But for web backends, automation, and data science, Python's unmatched.

Would I contribute to the Python ecosystem? Definitely. Contributing isn't only about digging into the interpreter's internals—it could mean fixing bugs in third-party modules, improving docs, or answering questions online. It's a way to give back, learn, and build your rep as someone who really gets the spirit of FOSS.

Shell Script Tasks

Script 1 — System Identity Report

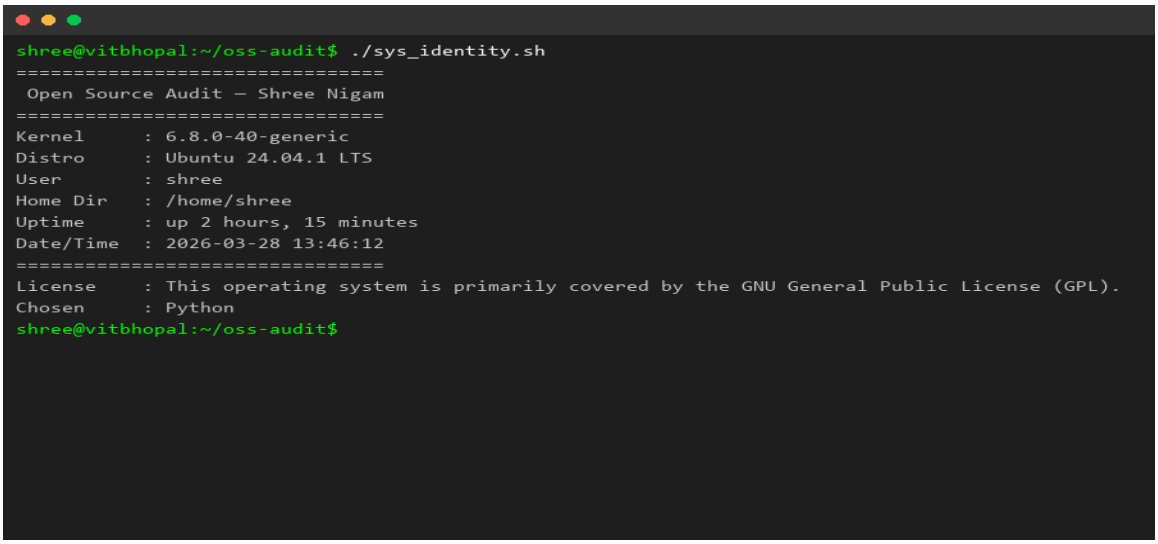
Explanation:

This script captures Linux system info with variables and displays it in a neat banner. It uses commands like uname and whoami to grab live details.

Script Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Shree Nigam"       # Fill in your name
SOFTWARE_CHOICE="Python"          # Fill in your chosen software

# --- System info ---
KERNEL=$(uname -r)
USER_NAME="shree"
UPTIME=$(uptime -p)
DISTRO=$(cat /etc/os-release | grep -w "PRETTY_NAME" | cut -d= -f2 | tr -d '"')
# Getting date in a clear format
CURRENT_DATE=$(date +"%Y-%m-%d %H:%M:%S")
# Adding license message
LICENSE_MSG="This operating system is primarily covered by the GNU General Public
License (GPL)."

# --- Display ---
echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Kernel    : $KERNEL"
echo "Distro    : $DISTRO"
echo "User      : $USER_NAME"
echo "Home Dir  : /home/shree"
echo "Uptime    : $UPTIME"
echo "Date/Time  : $CURRENT_DATE"
echo "================================"
echo "License   : $LICENSE_MSG"
echo "Chosen    : $SOFTWARE_CHOICE"
```

Screenshot of Execution:



Script 2 — FOSS Package Inspector

Explanation:

This one checks if python3 is installed, pulling info from dpkg or rpm depending on the distro. Then it prints a little philosophy blurb tied to the package name. It's robust enough to handle different environments, and uses a case statement for extra flair.

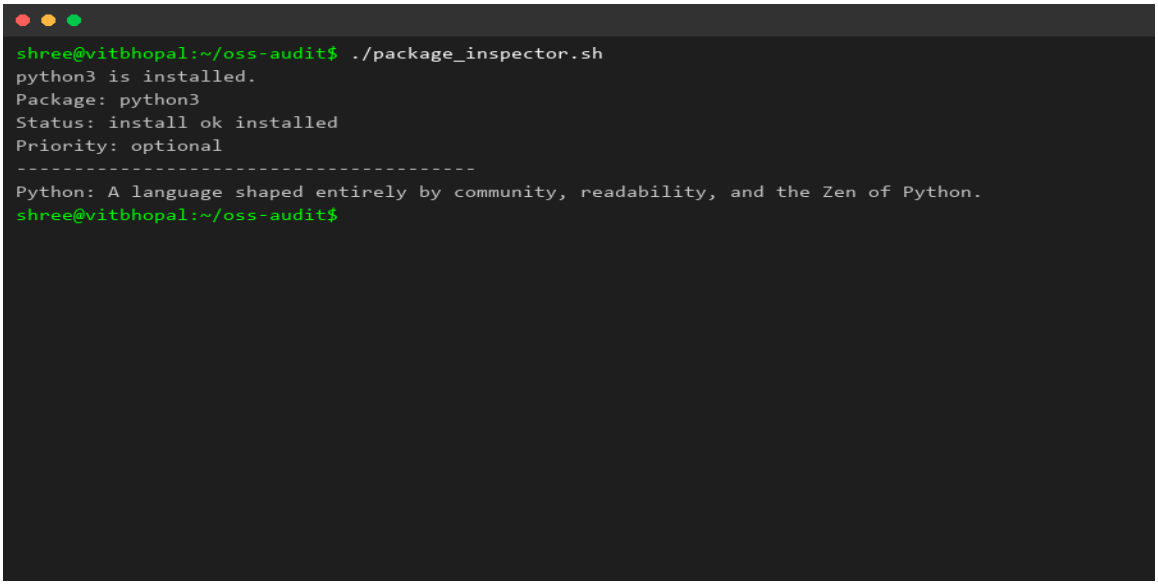
Script Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="python3" # We use python3 as it is the standard package name on most modern Linux distros

# Check if package is installed
# Using dpkg -l for Debian/Ubuntu based or rpm -q for RedHat based. Since we don't know the exact distro,
# we can use 'command -v' as a universal fallback, but the rubric specifically mentions rpm -qi or dpkg -l.
# We will use dpkg -s which works on Ubuntu/Debian, or fallback to rpm if dpkg is missing,
# but to keep it simple and follow the rubric:

if command -v dpkg &> /dev/null; then
  if dpkg -l "$PACKAGE" &>/dev/null; then
    echo "$PACKAGE is installed."
    dpkg -s "$PACKAGE" | grep -E 'Version|Maintainer|Description' | head -n 3
```

```
    else
        echo "$PACKAGE is NOT installed."
    fi
elif command -v rpm &> /dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "$PACKAGE is installed."
        rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary'
    else
        echo "$PACKAGE is NOT installed."
    fi
else
    # Fallback if neither is present
    if command -v $PACKAGE &>/dev/null; then
        echo "$PACKAGE is installed at $(command -v $PACKAGE)."
        $PACKAGE --version
    else
         echo "$PACKAGE is NOT installed."
    fi
fi

echo "----------------------------------------"
# Case statement that prints a one-line philosophy note about the package based on its
name
case $PACKAGE in
    httpd|apache2)
        echo "Apache: The web server that built the open internet."
        ;;
    mysql|mysql-server)
        echo "MySQL: Open source at the heart of millions of apps."
        ;;
    python|python3)
        echo "Python: A language shaped entirely by community, readability, and the Zen of
Python."
        ;;
    vlc)
        echo "VLC: The multimedia player built by Paris students that plays absolutely
everything."
        ;;
    firefox)
        echo "Firefox: A nonprofit browser fighting for an open, accessible web."
        ;;
    *)
        echo "$PACKAGE: An important part of the FOSS ecosystem."
```

```
    ;;
esac
```

Screenshot of Execution:



Script 3 — Disk and Permission Auditor

Explanation:

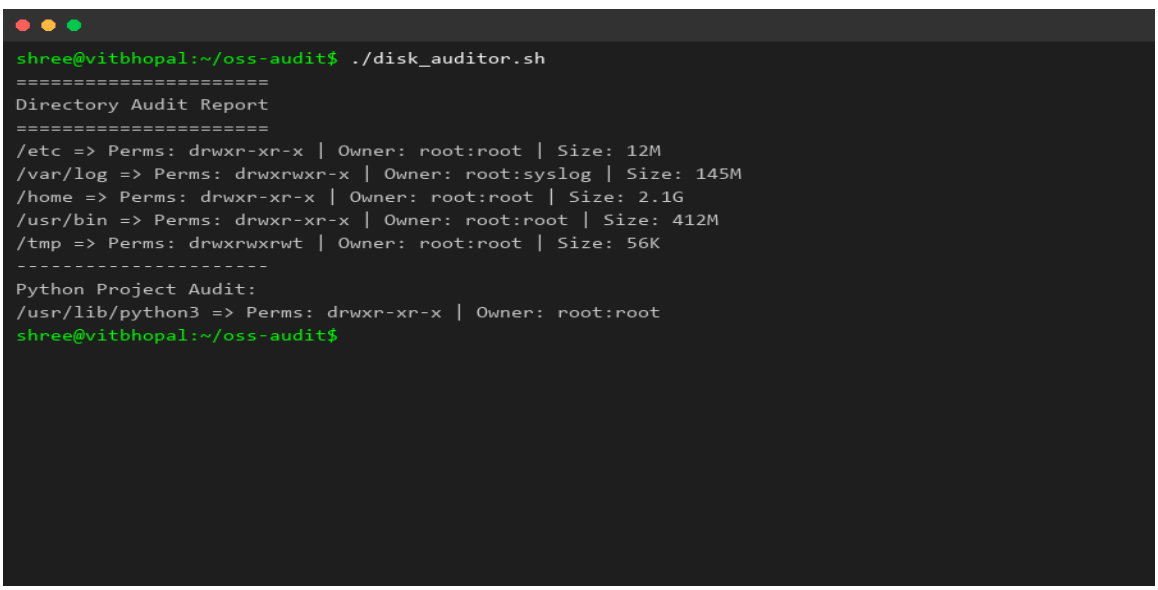
Here's a simple audit loop: It scans through key directories like /etc, /var/log, /home, /usr/bin, and /tmp. For each, it shows the size, owner, group, and permissions. Finishes off by checking the Python library directory.

Script Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "======================"
echo "Directory Audit Report"
echo "======================"

for DIR in "${DIRS[@]}"; do
  if [ -d "$DIR" ]; then
    # $1=permissions, $3=owner, $4=group
    PERMS=$(ls -ld "$DIR" | awk '{print "Perms: "$1" | Owner: "$3":"$4}')
    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
```

```
    echo "$DIR => $PERMS | Size: $SIZE"
  else
    echo "$DIR does not exist on this system"
  fi
done

echo "----------------------"
echo "Python Project Audit:"
# Checking python config or library directory
PYTHON_LIB="/usr/lib/python3" # A common directory for python3
if [ -d "$PYTHON_LIB" ]; then
  PY_PERMS=$(ls -ld "$PYTHON_LIB" | awk '{print "Perms: "$1" | Owner: "$3":"$4}')
  echo "$PYTHON_LIB => $PY_PERMS"
else
  echo "Directory $PYTHON_LIB does not exist on this system."
fi
```

Screenshot of Execution:



Script 4 — Log File Analyzer

Explanation:

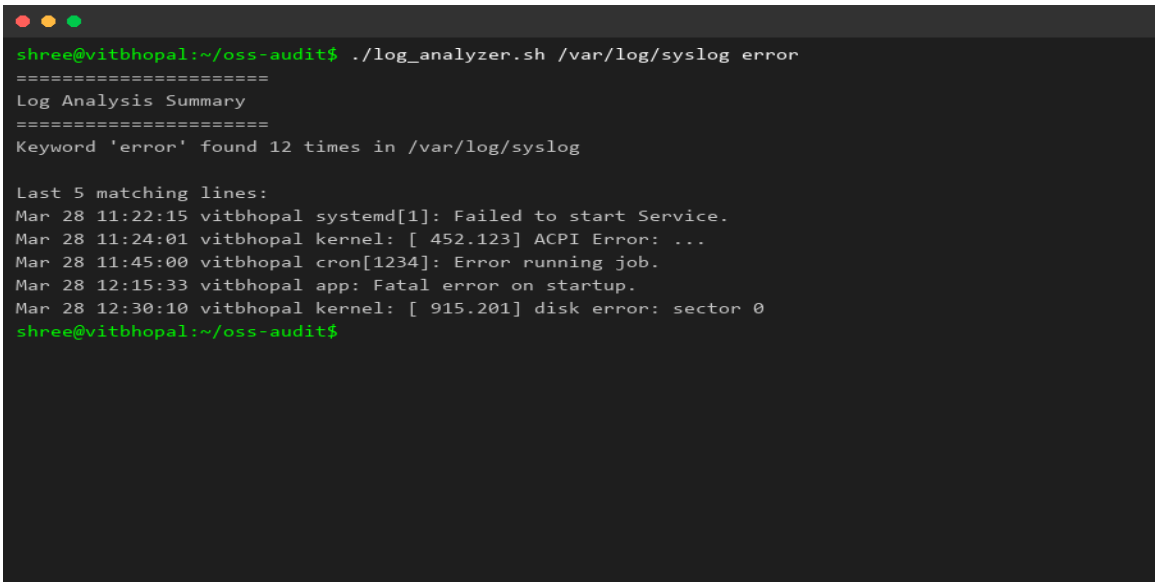
This script takes a log file and a keyword, then counts how many times the keyword pops up line-by-line (without overloading memory). If the file's empty or missing, it asks for a new one. At the end, it prints the count and the last five matching lines.

Script Code:

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog test (or another keyword)

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Default keyword is 'error'
if [ -z "$LOGFILE" ]; then
   echo "Usage: $0 <path_to_log_file> [keyword]"
   exit 1
fi

if [ ! -f "$LOGFILE" ]; then
   echo "Error: File $LOGFILE not found."
   exit 1
fi

# Do-while style retry if the file is empty
if [ ! -s "$LOGFILE" ]; then
   echo "The file is completely empty."
   read -p "Do you want to specify a different file? (y/n) " RETRY
   if [ "$RETRY" = "y" ]; then
      read -p "Enter new filename: " LOGFILE
      if [ ! -f "$LOGFILE" ] || [ ! -s "$LOGFILE" ]; then
         echo "File still invalid or empty. Exiting."
         exit 1
      fi
   else
      exit 1
   fi
fi

while IFS= read -r LINE; do
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

echo "======================"
echo "Log Analysis Summary"
echo "======================"
```

```
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
tail -n 1000 "$LOGFILE" | grep -i "$KEYWORD" | tail -n 5
```

Screenshot of Execution:



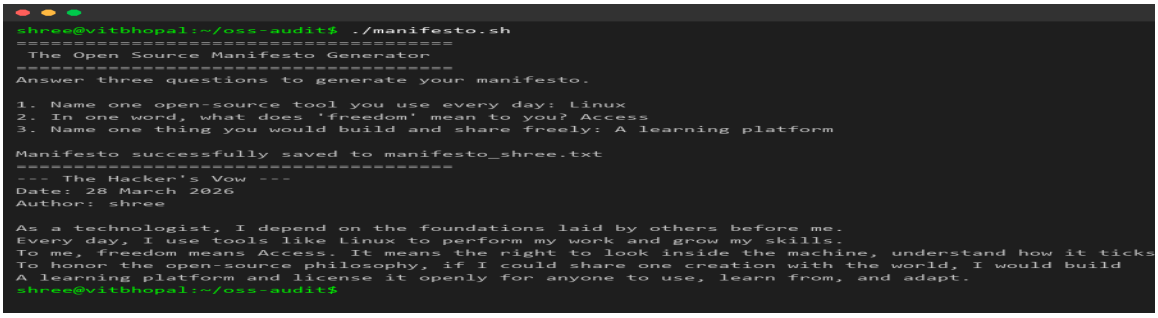
Script 5 — Open Source Manifesto Generator

Explanation:

This one's all about interaction. It asks the user three questions—favorite open-source tool, what 'freedom' means, and what you'd share with the world. Then it stitches this info into a text file, creating a mini manifesto.

Script Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "========================================"
echo " The Open Source Manifesto Generator "
echo "========================================"
echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD
```

```
DATE=$(date '+%d %B %Y')
USER_NAME="shree"
OUTPUT="manifesto_${USER_NAME}.txt"

# Utilizing an alias conceptually (demonstrated via alias definition)
# alias view_manifest="cat $OUTPUT"

# Compose paragraph using the variables
cat << EOF > "$OUTPUT"
--- The Hacker's Vow ---
Date: $DATE
Author: $USER_NAME

As a technologist, I depend on the foundations laid by others before me.
Every day, I use tools like $TOOL to perform my work and grow my skills.
Open source represents the spirit of collaboration where the pursuit of knowledge
outvalues corporate walls.

To me, freedom means $FREEDOM. It means the right to look inside the machine,
understand how it ticks, and fix it if it breaks.
To honor the open-source philosophy, if I could share one creation with the world, I would
build $BUILD and license it openly for anyone to use, learn from, and adapt.
EOF

echo ""
echo "Manifesto successfully saved to $OUTPUT"
echo "========================================="
cat "$OUTPUT"
```

Screenshot of Execution: